TECMILENIO

Gestión de Proyectos de Inteligencia Artificial

Reproducción de modelo local

Integración funcional, casos de error y verificación de Docker

Institución	Universidad Tecmilenio
Materia	Gestión de Proyectos de Inteligencia Artificial
Actividad	Proyecto Fase II
Proyecto	Evaluación experimental de riesgo crediticio
Alumno	Miguel Ángel Alejandre Pérez
Docente	Luis Ariel Vazquez Piña
Periodo	08 Noviembre de 2026

Repositorio: github.com/miguelalejandreperez-rgb/riesgo-crediticio-fase-II

1. Objetivo y criterios de evaluación

El objetivo fue comprobar el funcionamiento del prototipo web de riesgo crediticio, desde la recepción de entradas hasta la respuesta del modelo, y verificar que el contenedor y la API pudieran ejecutarse de forma reproducible.

2. Sistema y entorno de prueba

Elemento	Configuración / observación
Caso de uso	Puntuación experimental de incumplimiento crediticio.
Modelo	HistGradientBoostingClassifier con imputación de mediana.
Variables	19 entradas: límite, seis estados de pago, seis facturaciones y seis pagos.
API	FastAPI, rutas /api/health, /api/modelo, /api/predecir y /docs.
Infraestructura	Docker ejecutado en GitHub Codespaces, puerto 8000 privado.
Control de versiones	Repositorio GitHub; rama main; commit 8726777 en validación automática.
Automatización	Workflow docker-validation #1, estado Success, 1 min 6 s.

El modelo procede de un notebook previo de la Actividad 6. El conjunto de datos de origen contiene 30,000 registros de crédito de Taiwán (2005). Se empleó un umbral experimental de alerta de 0.14 en el modelo verificado. El puntaje obtenido con predict_proba no está calibrado como probabilidad de riesgo real.

3. Procedimiento de validación

La comprobación combinó el arranque del contenedor en Codespaces, solicitudes HTTP desde terminal, operación manual del formulario, pruebas en Swagger UI y una ejecución de GitHub Actions. Los resultados se registraron a partir de las respuestas visibles en pantalla.

3.1 Matriz de pruebas realizadas

ID	Capa	Caso	Resultado observado	Estado
P01	Docker	Construcción y arranque en Codespaces	Imagen creada y aplicación inicial comprobada	Conforme
P02	Servicio	GET /api/health	HTTP 200; estado listo y modelo identificado	Conforme
P03	API real	scripts/smoke_docker.sh	Puntaje 0.181737; umbral 0.14; alerta true	Conforme
P04	Formulario	Entrada de ejemplo completa	27.60 %; alerta activada	Conforme
P05	Estabilidad	Repetir entrada sin modificarla	27.60 % en la repetición	Conforme
P06	Frontend	LIMIT_BAL = 0	Bloqueado: mínimo 0.01	Conforme
P07	Frontend	LIMIT_BAL = -1000	Bloqueado: mínimo 0.01	Conforme
P08	API	LIMIT_BAL = -1000	HTTP 422; gt 0	Conforme
P09	API válida	JSON con 19 variables	HTTP 200; puntaje 0.27602; alerta true	Conforme
P10	Consistencia	Comparar formulario y API	0.27602 ≈ 27.60 %	Conforme
P11	Campo faltante	Omitir BILL_AMT6	HTTP 422; Field required	Conforme
P12	JSON incorrecto	Prueba de valor extremo con coma omitida	HTTP 422; json_invalid; no evaluó el límite	Inconclusa para extremo
P13	Automatización	GitHub Actions workflow #1	Success; job docker-validation	Conforme

3.2 Resultados de predicción

El formulario arrojó una puntuación del 27.60 % para la entrada de ejemplo. La solicitud equivalente ejecutada directamente en POST /api/predecir devolvió puntaje_incumplimiento = 0.27602, umbral_alerta = 0.14 y alerta = true. La diferencia aparente se explica por el redondeo visual del porcentaje, por lo que ambas interfaces entregaron una salida consistente.

La prueba automática de humo utilizó otra entrada y devolvió una puntuación de 0.181737, también superior al umbral. Ambos resultados corresponden a datos de demostración y no deben interpretarse como probabilidades calibradas de incumplimiento.

3.3 Validaciones negativas

El navegador bloqueó un límite de crédito igual a cero y uno de -1000, indicando que el valor mínimo aceptado por el formulario es 0.01. En la API, LIMIT_BAL = -1000 produjo HTTP 422 con la regla de valor estrictamente positivo. La omisión del campo BILL_AMT6 también devolvió HTTP 422 y el detalle Field required.

Diferencia detectada La interfaz utiliza un mínimo de 0.01 y el backend acepta cualquier valor mayor que cero. Se recomienda homologar ambas reglas y documentar el rango de negocio admitido.

3.4 Caso extremo no concluido

Se intentó una solicitud con LIMIT_BAL = 1,000,000,000, pero el servidor respondió HTTP 422 con tipo json_invalid y mensaje Expecting ',' delimiter. La petición no llegó a la validación numérica del modelo. Por tanto, el comportamiento frente a ese límite alto permanece sin evaluar y no se presenta como una prueba superada.

4. Verificación de Docker y automatización

La construcción de la imagen riesgo-crediticio-faseii:1.0 finalizó correctamente en Codespaces. El contenedor se recuperó en una sesión posterior y GET /api/health respondió HTTP 200 con el modelo HistGradientBoostingClassifier + imputación mediana. La aplicación también respondió en la ruta / y permitió realizar inferencias desde el navegador.

El flujo manual de GitHub Actions Construir y verificar Docker con modelo versionado se ejecutó sobre main y el commit 8726777. Su job docker-validation terminó con estado Success en 59 segundos y la ejecución completa duró 1 minuto y 6 segundos. La pantalla también mostró advertencias sobre deprecación de Node.js 20 y cambios futuros del runner Ubuntu, sin impedir el resultado exitoso.

5. Hallazgos y puntos pendientes

Tema	Interpretación / acción
Ámbito de ejecución	Docker fue validado en Codespaces, no físicamente en Windows 11.
Prueba extrema	Reenviar JSON bien formado con LIMIT_BAL alto y registrar si devuelve 200 o 422.
Reglas de entrada	Alinear mínimo del frontend (0.01) con la regla gt 0 del backend.
Dependencias de CI	Revisar acciones de GitHub para resolver los avisos de compatibilidad futuros.
Modelo de riesgo	No está calibrado ni validado externamente; prohibir datos reales y uso decisional.

6. Conclusión

Las comprobaciones realizadas acreditan que el prototipo puede ejecutarse mediante Docker en un entorno remoto de desarrollo, atender peticiones REST y mostrar resultados coherentes entre la API y el formulario. El backend controla valores inválidos y campos obligatorios; GitHub Actions reprodujo satisfactoriamente la construcción y las pruebas automatizadas.

El alcance verificado corresponde a un prototipo académico. Antes de cualquier despliegue público deben completarse las pruebas extremas, la homogeneización de reglas de entrada y la implementación de controles de seguridad. La evaluación del comportamiento crediticio real requeriría, además, calibración, validación temporal y revisión de sesgos sobre una población apropiada.

Referencias y fuentes

- Repositorio del proyecto: https://github.com/miguelalejandreperez-rgb/riesgo-crediticio-fase-ll
- Notebook de origen: original/Actividad_6_Riesgo_Crediticio.ipynb
- Universidad Tecmilenio. Rúbrica de evaluación, Proyecto Fase II.
- GitHub Actions: ejecución manual del flujo validate-docker.yml, commit 8726777.
- Dataset de origen: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset

7. Evidencias de ejecución

Las figuras siguientes corresponden a las capturas de las pruebas realizadas en Codespaces, Swagger UI y GitHub Actions. Se conservan los códigos HTTP, mensajes y resultados relevantes.

```
@miguelalejandreperez-rgb ➜/workspaces/riesgo-crediticio-fase-ll (main) $ bash scripts/start_codespaces.sh
 => => extracting sha256:7efbce397249e9202b070b8e638baff71160948c640b9a44f2226419a71f410d                    0.7s
 => => extracting sha256:bbe7053e6def9842d628414346e26f25e499463adb6ca48c0c8932e036e44000                    0.0s
 => [internal] load build context                                                                            0.1s
 => => transferring context: 113.42kB                                                                        0.0s
 => [2/7] WORKDIR /srv/proyecto                                                                              2.8s
 => [3/7] COPY requirements-runtime.txt .                                                                    0.0s
 => [4/7] RUN python -m pip install --no-cache-dir -r requirements-runtime.txt     && useradd --create-home --uid 10001 appuser   28.6s
 => [5/7] COPY app/ ./app/                                                                                   0.3s
 => [6/7] COPY artifacts/modelo_riesgo.joblib artifacts/metadata.json ./artifacts/                           0.1s
 => [7/7] RUN chown -R appuser:appuser /srv/proyecto                                                         0.3s
 => exporting to image                                                                                      26.7s
 => => exporting layers                                                                                      9.4s
 => => exporting manifest sha256:82fc5386bb6d37f3395ff5acb038e8a125ece94100e0ebd8d720d0248c988245            0.0s
 => => exporting config sha256:7e563e34172cdfcc295985ca3e347093d35b321b33ed1afd47158100c72bfa34              0.0s
 => => exporting attestation manifest sha256:8f5c9f04ea1f443489f889253d98b5930fe7399838264c3b4ae84327bf1d9ac1  0.0s
 => => exporting manifest list sha256:a73d9ee2880f82cd2a33e5d76d112a405cc3f50b6cbfea0d3c09c091a58644d3       0.0s
 => => naming to docker.io/library/riesgo-crediticio-faseii:1.0                                              0.0s
 => => unpacking to docker.io/library/riesgo-crediticio-faseii:1.0                                           17.3s
1ca4927b3c6dc9fcaca62d577911e50c5c67e1da5d6b513268c8e9996e32616c
Aplicacion verificada en http://localhost:8000
En Codespaces, abre el puerto 8000 desde la pestana PORTS (visibilidad Private).
@miguelalejandreperez-rgb ➜/workspaces/riesgo-crediticio-fase-ll (main) $
```

Figura 1. Construcción de imagen Docker y comprobación inicial del servicio.



Figura 2. Resultado del formulario: 27.60 % y alerta activada.

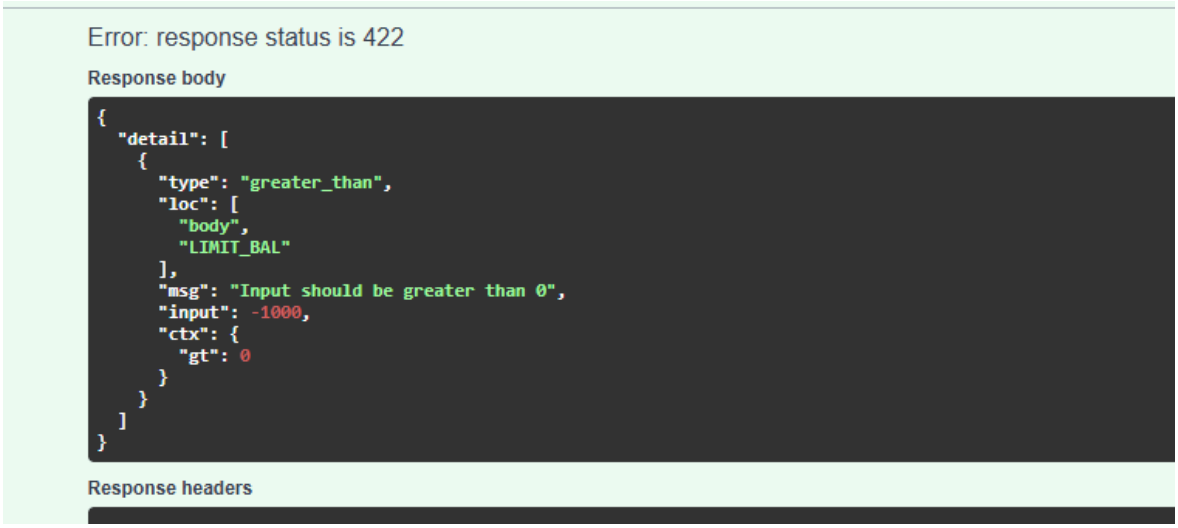


Figura 3. API: rechazo de LIMIT_BAL = -1000 con código HTTP 422.



Figura 4. API: respuesta HTTP 200 con puntaje 0.27602 y alerta true.

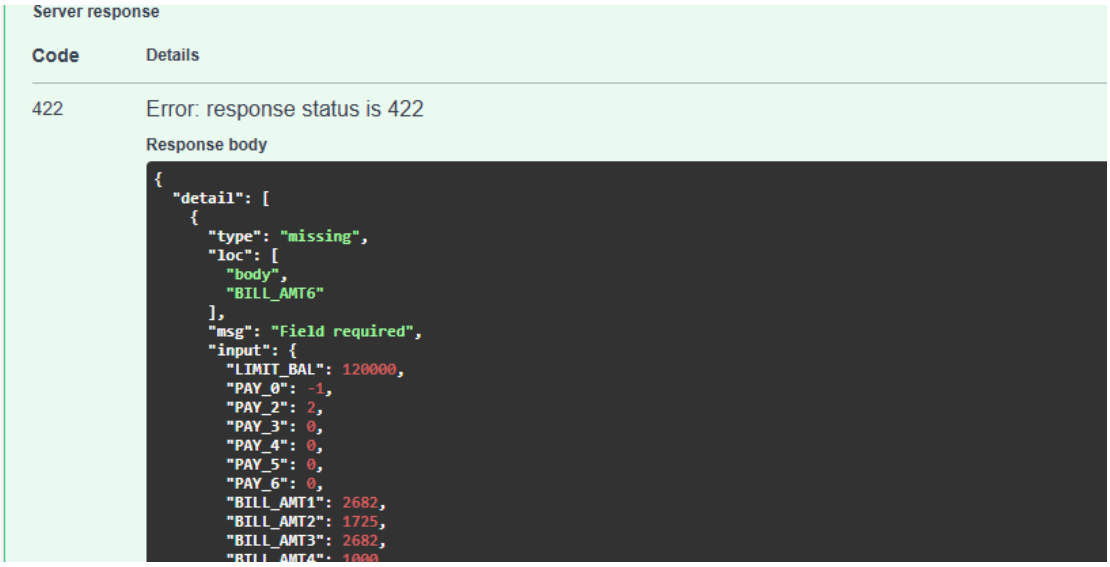


Figura 5. Validación de campo faltante BILL_AMT6 mediante HTTP 422.



Figura 6. GitHub Actions: ejecución #1 concluida con estado Success.



Figura 7. Error de sintaxis JSON en la prueba extrema; resultado no concluyente.